

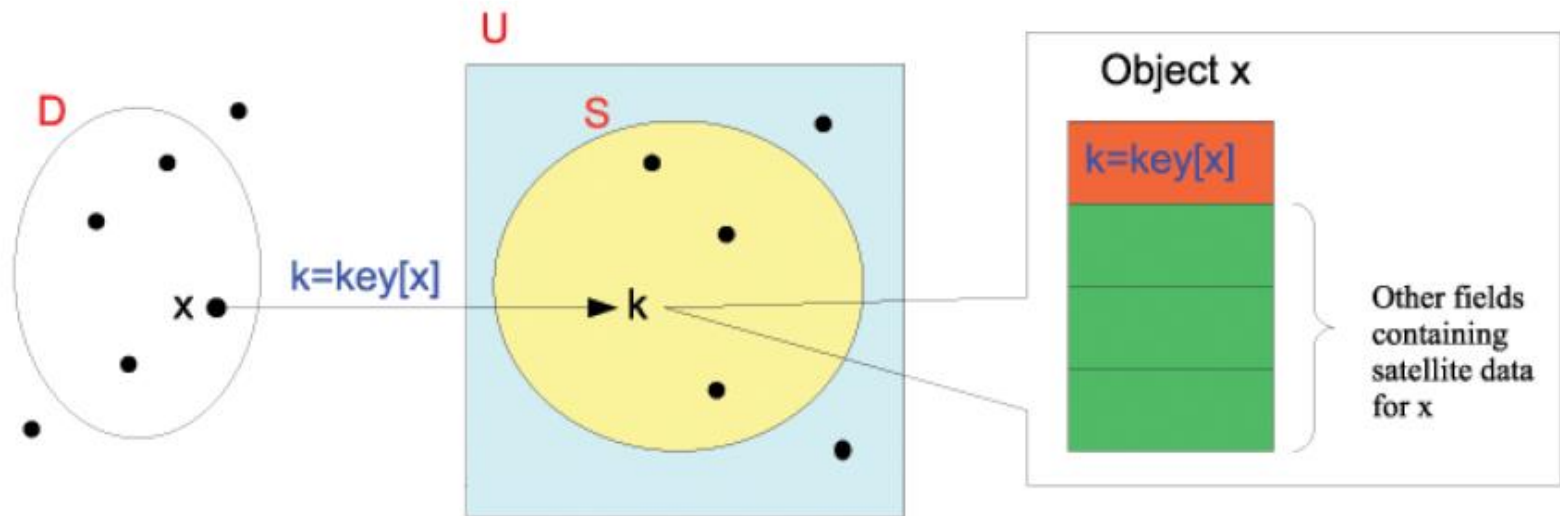


Hash table

Martedì 31 Marzo

Implementare un dizionario

- + Agli oggetti del dizionario si accede tramite chiavi



Implementazioni Dizionario

Tempo richiesto dall'operazione più costosa:

- Liste $O(n)$
- Alberi di ricerca non bilanciati $O(n)$
- Alberi di ricerca bilanciati $O(\log n)$
- **Tabelle hash** $O(1)$

Array

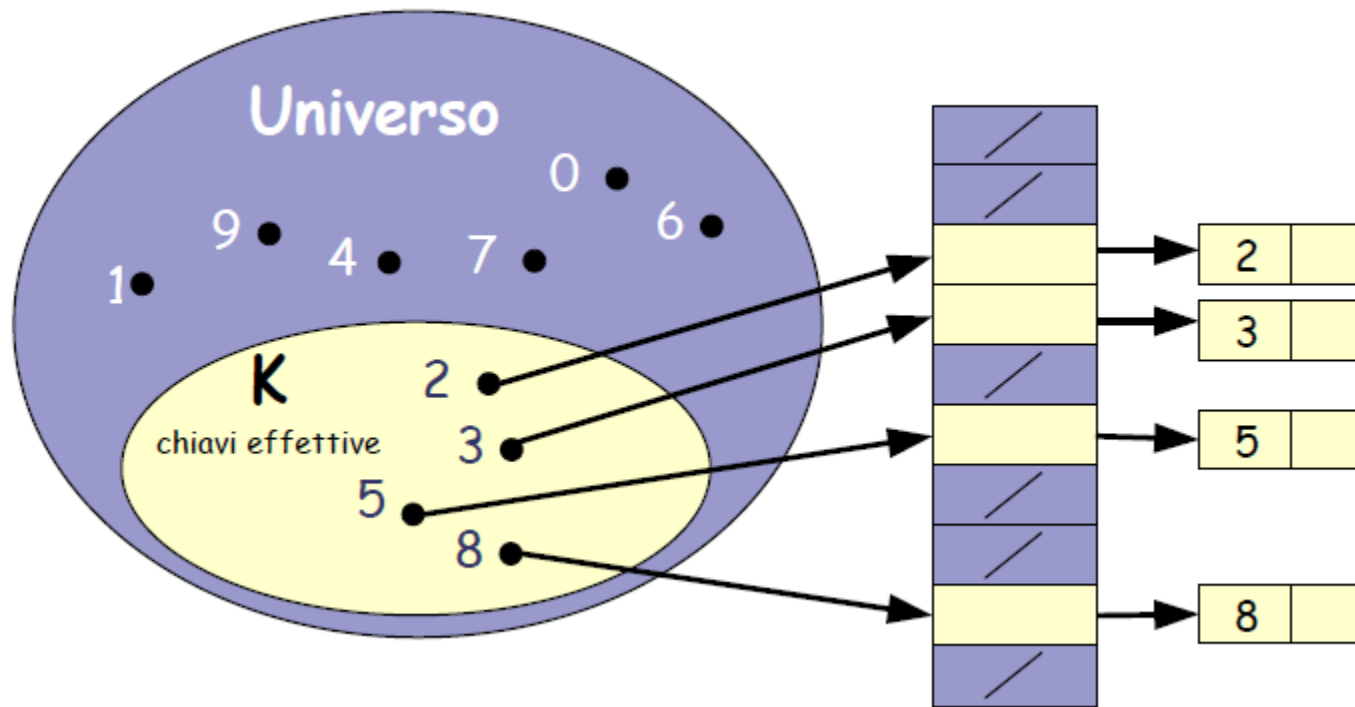
- + Se le chiavi sono interi piccoli, allora un array può essere usato per implementare un dizionario. Infatti, basta interpretare una chiave come l'indice di un array così da memorizzare il valore associato con la chiave K nella posizione K dell'array.
- + Si parla in questo caso di Tabelle ad accesso diretto.
- + Le tabelle hash estendono quest'approccio per tipologie di chiavi più complesse.

Tabelle ad accesso diretto

Sono dizionari basati sulla proprietà di accesso diretto alle celle di un array

- dizionario memorizzato in array v di m celle
- a ciascun elemento è associata una chiave intera nell'intervallo $[0, m-1]$
- elemento con chiave k contenuto in $v[k]$
- al più $n \leq m$ elementi nel dizionario

La tabella ad accesso diretto



Pregi e difetti

Pregi:

- Tutte le operazioni richiedono tempo $O(1)$

Difetti:

- Le chiavi devono essere necessariamente interi in $[0, m-1]$
- Lo spazio utilizzato è proporzionale ad m , non al numero n di elementi: può esserci grande spreco di memoria!

Fattore di carico

- Misuriamo il grado di riempimento di una tabella introducendo il fattore di carico:
 $\alpha = N/M$
dove M è la dimensione della tabella e N è il numero di chiavi effettivamente utilizzate
- Esempio: tabella con nomi di studenti indicizzati da numeri di matricola a 6 cifre

$$N = 100, M = 10^6, \alpha = 0,0001 = 0,01\%$$

Eccessivo spreco di memoria.

E se le chiavi non sono interi?

Tabelle hash

Per ovviare agli inconvenienti delle tabelle ad accesso diretto ne consideriamo un'estensione: le **tabelle hash**

- Chiavi prese da un universo totalmente ordinato U (possono non essere numeri)
- Funzione hash: $h: U \rightarrow [0, m-1]$
(funzione che trasforma chiavi in indici)
- Elemento con chiave k in posizione $v[h(k)]$

Un dizionario come *tabella hash*

- ⦿ Idea: Dimensionare la tabella in base al numero di elementi attesi ed utilizzare una speciale funzione (funzione hash) per indicizzare la tabella
- ⦿ Questo tipo di organizzazione consente di implementare le tre operazioni fondamentali con un numero molto contenuto di accessi indipendente dalla dimensione della tabella.

Un dizionario come *tabella hash*

- Una ricerca basata su hashing è completamente diversa da quella basata su confronti:

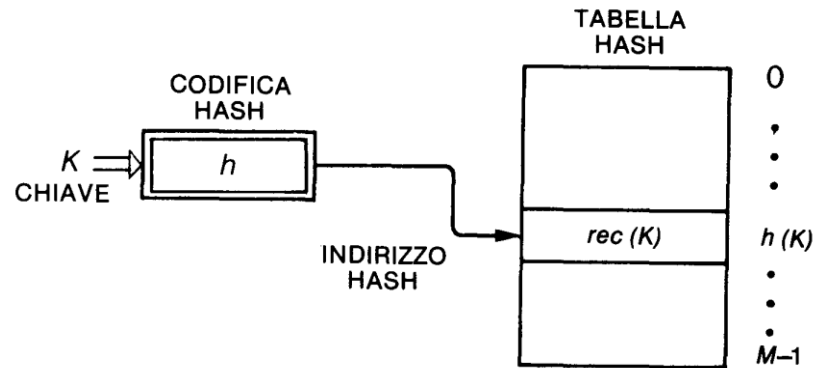
invece che spostarci nella struttura dati in funzione dell'esito dei confronti fra chiavi, cerchiamo di accedere agli elementi della tabella in modo diretto tramite operazioni aritmetiche che trasformano le chiavi in indirizzi della tabella.

- E' necessario determinare la **funzione di hash** che trasforma una chiave in un indirizzo della tabella.*

La funzione *hash*

- Un indirizzo hash è un possibile valore dell'indice di un vettore di elementi destinati ad ospitare gli elementi della tabella.
- Una funzione hash ideale è facilmente calcolabile e approssima una funzione casuale: per ogni valore di input, i possibili valori di output dovrebbero essere in qualche senso equiprobabili.

Un dizionario come *tabella hash*



Con l'hashing, un elemento con chiave k viene memorizzato nella cella $h(k)$.

Pro: riduciamo lo spazio necessario per memorizzare la tabella

Contra: perdiamo la corrispondenza tra chiavi e posizioni in tabella

Funzione hash perfetta

- La situazione ideale si ha quando la funzione h è iniettiva:

$$\text{per ogni } K, K' \in S: \quad K \neq K' \Rightarrow h(K) \neq h(K')$$

- In questo caso ogni chiave ha il suo indirizzo hash distinto da quello di tutte le altre chiavi: ogni elemento della tabella può essere raggiunto con un solo accesso (**funzione hash perfetta**).

Semplice implementazione

classe TavolaHashPerfetta **implementa** Dizionario:

dati:

$$S(m) = \Theta(m)$$

un array v di dimensione $m \geq n$ in cui $v[h(k)] = e$ se c'è un elemento e con chiave $k \in U$ nel dizionario, e $v[h(k)] = \text{null}$ altrimenti. La funzione $h : U \rightarrow \{0, \dots, m - 1\}$ è una funzione hash perfetta calcolabile in tempo $O(1)$.

operazioni:

$\text{insert}(\text{elem } e, \text{chiave } k)$
 $v[h(k)] \leftarrow e$

$$T(n) = O(1)$$

$\text{delete}(\text{chiave } k)$
 $v[h(k)] \leftarrow \text{null}$

$$T(n) = O(1)$$

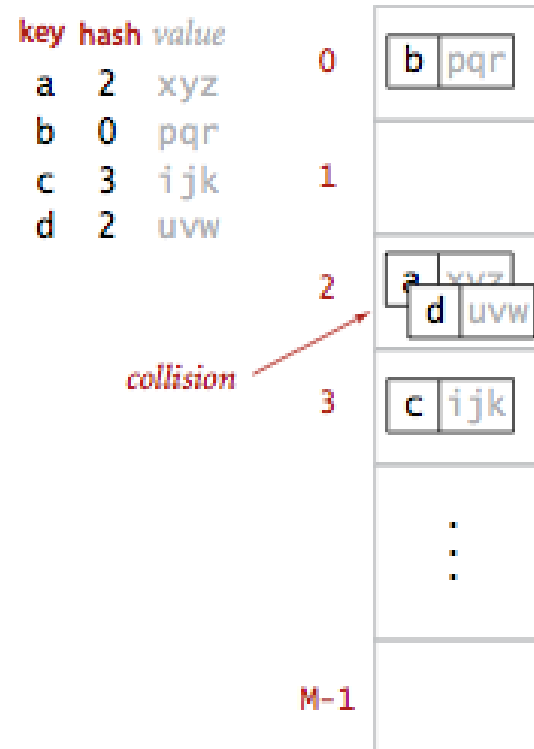
$\text{search}(\text{chiave } k) \rightarrow \text{elem}$
return $v[h(k)]$

$$T(n) = O(1)$$

tabella hash: collisioni

- ⦿ Se la funzione hash non è perfetta, allora è possibile che si verifichino «collisioni», ovvero elementi distinti hanno lo stesso valore della funzione hash.
- ⦿ D'altra parte questo si può certamente verificare se la dimensione dell'array (dizionario) è minore del numero delle possibili chiavi.

Esempio



Hashing: the crux of the problem

tabella hash: *come affrontare le collisioni?*

- + Le chiavi possibili sono in genere moltissime e non note a priori (per esempio le variabili che un utente inserirà in un programma, o i cognomi dei clienti di una ditta), quindi si ha $|\text{Insieme delle chiavi}| \gg M$.
- + Dunque è inevitabile che nell'applicazione si presentino due o più chiavi $K_1, K_2 \dots$ con lo stesso indirizzo hash, cioè $h(K_1)=h(K_2)=\dots$

Nasce quindi un problema di *collisioni*: solo una delle chiavi (tipicamente la prima che si è presentata, diciamo K_1) potrà essere allocata in $A[h(K_1)]$, e le altre saranno poste altrove.

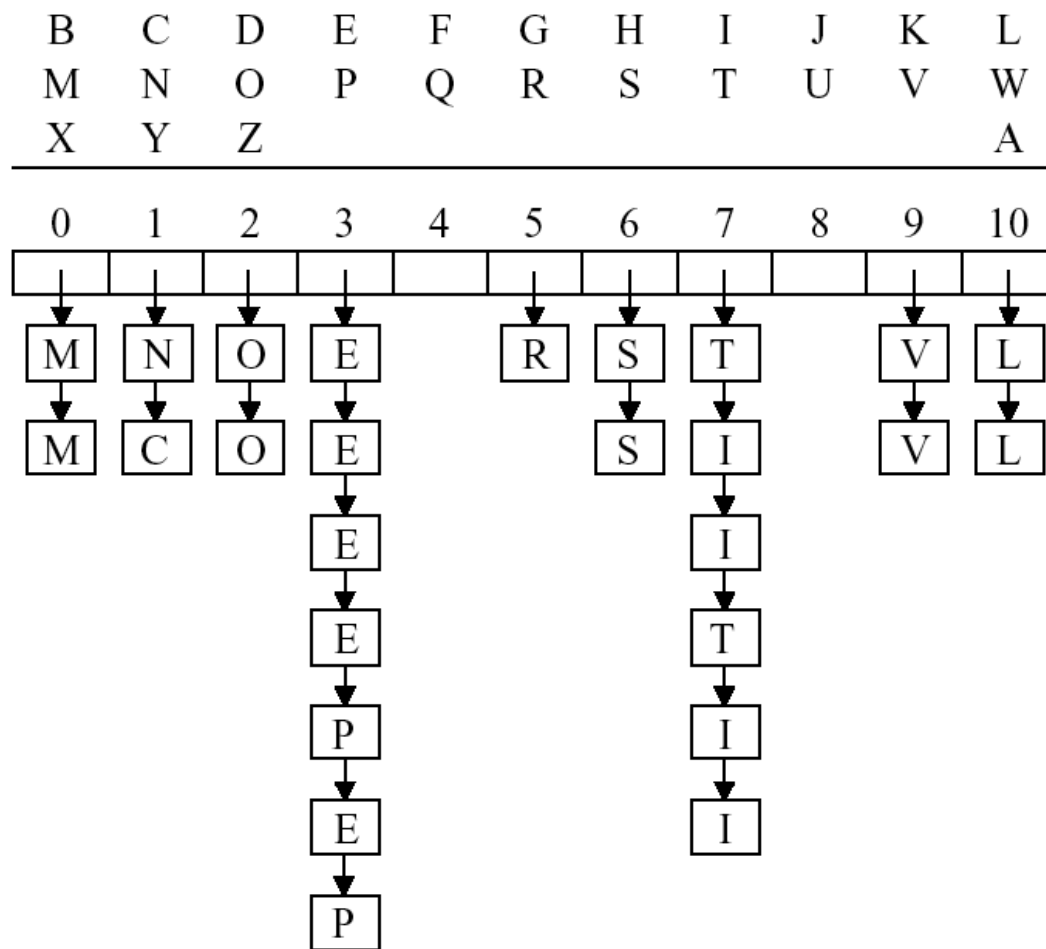
- + Dobbiamo quindi affrontare tre problemi:
 - 1) come si sceglie la dimensione M ;
 - 2) come si calcola la funzione h ;
 - 3) come si risolvono le collisioni.

Risoluzione delle collisioni

Nel caso in cui non si possano evitare le collisioni, dobbiamo trovare un modo per risolverle. Due metodi classici sono i seguenti

1. **Liste di collisione.** Gli elementi sono contenuti in liste esterne alla tabella: $v[i]$ punta alla lista degli elementi tali che $h(k)=i$
2. **Indirizzamento aperto.** Tutti gli elementi sono contenuti nella tabella: se una cella è occupata, se ne cerca un'altra libera

Liste di collisione (Separate chaining)



Esempio di tabella hash basata su liste di collisione contenente le lettere della parola:

**PRECIPITEVOLISSI
MEVOLMENTE**

Uniformità delle funzioni hash

Per ridurre la probabilità di collisioni, una buona funzione hash dovrebbe essere in grado di distribuire in modo uniforme le chiavi nello spazio degli indici della tabella

Questo si ha ad esempio se la funzione hash gode della proprietà di uniformità semplice

Uniformità semplice

Sia $P(k)$ la probabilità che la chiave k sia presente nel dizionario e sia:

$$Q(i) = \sum_{k:h(k)=i} P(k)$$

la probabilità che, scegliendo una chiave k a caso, finisca nella cella i .

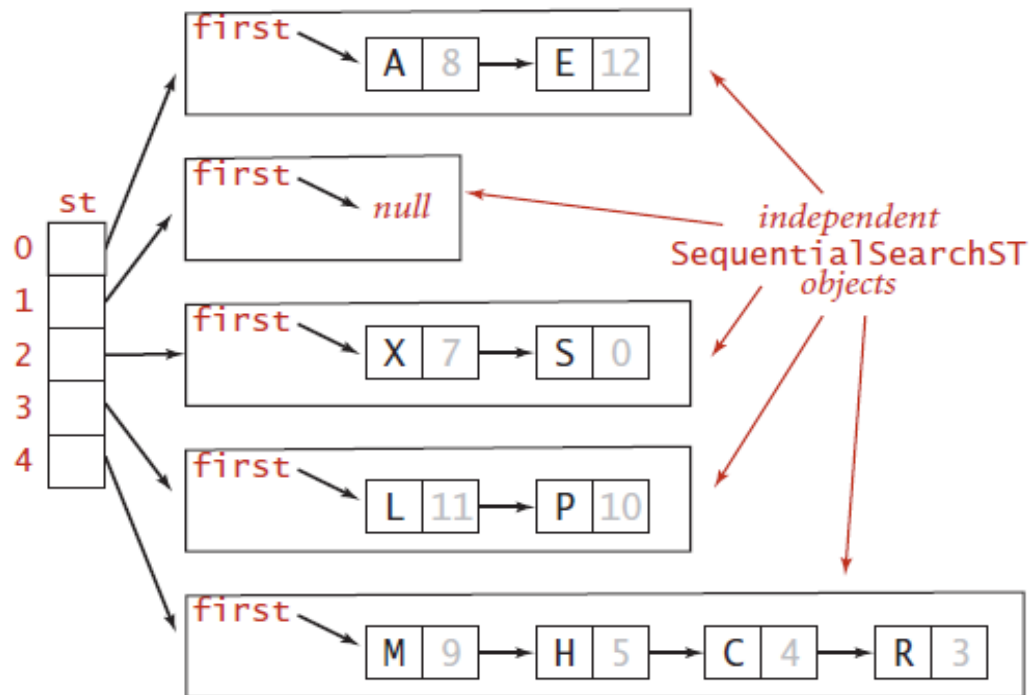
Una funzione hash gode della proprietà di uniformità semplice se, per ciascun indice i che varia in $0, \dots, m-1$, si ha:

$$Q(i) = \frac{1}{m}$$

Esempio

key hash value

S	2	0
E	0	1
A	0	2
R	4	3
C	4	4
H	4	5
E	0	6
X	2	7
A	0	8
M	4	9
P	3	10
L	3	11
E	0	12



Nel dettaglio

- + Separate chaining per la gestione delle collisioni
 - + Array di $M < N$ liste concatenate.
 - + Hash: da chiave k a intero i compreso tra 0 e $M - 1$;
 - + Insert: metti in testa alla i -esima lista (se non già presente);
 - + Search: ricerca lungo la i -esima lista.

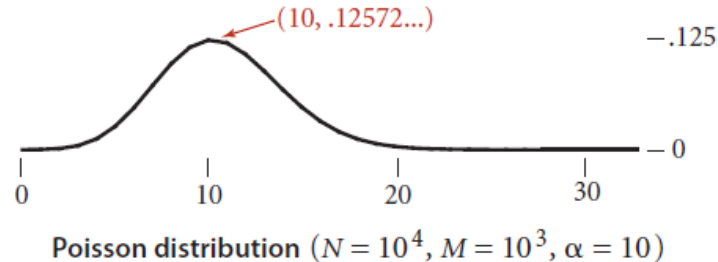
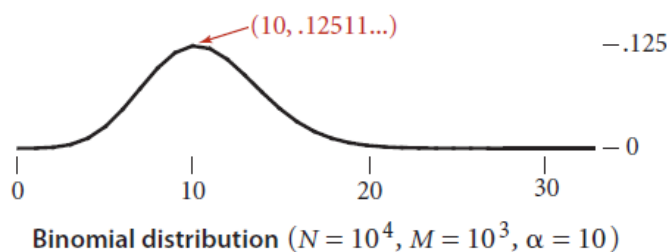
Quanto costano le operazioni di dizionario?

- + Dipendono dalla lunghezza delle liste.
- + Nell'ipotesi di hashing uniforme, la lunghezza media di ogni lista sarà $\alpha = N/M$, se N è il numero delle chiavi e M la dimensione dell'array.
- + Anche se tale ipotesi non c'è, il valore medio delle lunghezze è N/M (per es. $N+0+0+\dots+0)/M$.

Separate chaining: analisi

- + Qualsiasi sia la distribuzione delle $N > M$ chiavi, la lunghezza media di una lista è N/M , ma noi vogliamo che tutte le liste abbiano lunghezza simile.
- + TEOREMA: Sotto l'assunzione di uniformità dell'hashing, la probabilità che il numero delle chiavi in una lista stia in un intorno di N/M è vicina a 1.
- + Dimostrazione: Definito $\alpha = \frac{N}{M}$ il fattore di carico, scelte k chiavi tra N , ogni chiave ha probabilità $1/M$ di essere in una delle liste e ciascuno delle chiavi restanti $N-k$ ha probabilità $1-1/M$. La probabilità che una data lista contenga esattamente k chiavi è data dalla distribuzione binomiale, ovvero $\binom{N}{k} \left(\frac{1}{M}\right)^k \left(1 - \frac{1}{M}\right)^{N-k} = \binom{N}{k} \left(\frac{\alpha}{N}\right)^k \left(1 - \frac{\alpha}{N}\right)^{N-k}$ che, per piccoli valori di α , approssima una distribuzione di Poisson, la quale ha picco in $k = \alpha$ e decade rapidamente.

Si prova che la probabilità che una lista abbia più di t α chiavi è $\frac{(\frac{\alpha e}{t})^t}{e^{-\alpha}}$ (e.g., per $\alpha = 10$, la probabilità di avere più di 20 chiavi è inferiore a 1%).



- + Caso peggiore: è noto che, se α è costante, la lista più lunga cresce come $\log N / \log(\log N)$.

Operazioni di dizionario

- + Teorema: Nell'ipotesi di hashing uniforme semplice, in una tabella hash con liste di collisione, una ricerca senza successo richiede un tempo atteso $\Theta(1 + \alpha)$
Dim: Se l'elemento k non è stato ancora inserito, ogni lista ha la stessa probabilità di essere presa in considerazione. Se il costo per calcolare la funzione hash è costante, il risultato segue dal teorema precedente.
- + Nel caso di ricerca con successo di un elemento x , il numero degli elementi esaminati è il numero degli elementi inseriti nella lista di x dopo x , più 1. In questo caso si dimostra che è in media $\Theta(1 + \alpha)$
- + Se N è proporzionale a M , allora diventa costante.

Operazioni di dizionario

classe `TavolaHashListeColl` **implementa** `Dizionario`:

dati:

$$S(m, n) = \Theta(m + n)$$

un array v di dimensione m in cui ogni cella contiene un puntatore a una lista di coppie $(elem, chiave)$. Un elemento e con chiave $k \in U$ è nel dizionario se e solo se (e, k) è nella lista puntata da $v[h(k)]$, con $h : U \rightarrow \{0, \dots, m-1\}$ funzione hash con uniformità semplice calcolabile in tempo $O(1)$.

operazioni:

`insert(elem e, chiave k)`

$$T(n) = O(1)$$

aggiungi la coppia (e, k) alla lista puntata da $v[h(k)]$.

`delete(chiave k)`

$$T_{avg}(n) = O(1 + n/m)$$

rimuovi la coppia (e, k) nella lista puntata da $v[h(k)]$.

`search(chiave k) → elem`

$$T_{avg}(n) = O(1 + n/m)$$

se (e, k) è nella lista puntata da $v[h(k)]$, allora restituisci e , altrimenti restituisci `null`.

Come scegliere una buona funzione hash?

- + La funzione hash dipende molto dal tipo di chiave presa in considerazione

Hashing modulare

- Se le chiavi sono intere, $h(k)=k\%M$ (hashing modulare).
- Nel caso di hashing modulare, se si usa una rappresentazione in base 10 e $M=10^p$, $h(k)$ rappresenta le p cifre meno significative. Se uso la rappresentazione binaria devo evitare potenze di 2, per esempio 2^p poichè tutte le chiavi che hanno i primi p bit uguali avrebbero la stessa funzione hash.
- Una buona scelta sarebbe M un numero primo (non troppo vicino ad una potenza di 2) poichè disperde i valori in modo più uniforme.

key	hash (M = 100)	hash (M = 97)
212	12	18
618	18	36
302	2	11
940	40	67
702	2	23
704	4	25
612	12	30
606	6	24
772	72	93
510	10	25
423	23	35
650	50	68
317	17	26
907	7	34
507	7	22
304	4	13
714	14	35
857	57	81
801	1	25
900	0	27
413	13	25
701	1	22
418	18	30
601	1	19

Modular hashing

Esempi di funzioni hash (hash table di taglia M)

- Se le chiavi sono numeri reali casuali in $[0,1)$, la funzione hash è $h(k) = \lfloor kM \rfloor$
Se le chiavi sono distribuite in modo indipendente e uniforme allora la funzione soddisfa la condizione di hashing uniforme.
- Se le chiavi sono numeri reali casuali in $[s,t)$, allora $h(k) = \lfloor (k-s)/(t-s) * M \rfloor$
- Quest'approccio ha qualche difetto poichè attribuisce maggiore peso ai bit più significativi. Un modo per ovviare a questo problema è usare l'hashing modulare alla rappresentazione binaria della chiave (ciò è quello che fa Java)

Esempi di funzioni hash (hash table di taglia M)

- + Se le chiavi sono stringhe, si cerca di convertirle in un numero naturale interpretando i simboli come numeri in sistemi di numerazione in base diversa.
- + Per convertire una stringa in un numero naturale si può considerare la stringa come un numero in base A, esistono cioè A simboli diversi per ogni cifra di una stringa
- + E' possibile stabilire una conversione fra ogni simbolo e i numeri naturali (codifica ASCII ad esempio)
- + La conversione può essere fatta considerando il numero espresso in un sistema posizionale in base 128. Abbiamo bisogno però di limitare il valore finale.
 - ▶ $c a s a_{(128)} =$
 - ▶ $c_3 a_2 s_1 a_0_{(128)} =$
 - ▶ $'c' * 128^3 + 'a' * 128^2 + 's' * 128^1 + 'a' * 128^0 =$
 - ▶ $99 * 128^3 + 97 * 128^2 + 115 * 128^1 + 97 * 128^0 =$
 - ▶ $99 * 2097152 + 97 * 16384 + 115 * 128 + 97 * 1 = 209222113_{(10)}$

Esempi di funzioni hash (hash table di taglia M)

- + La funzione hash può essere:

```
int hash = 0;
```

```
for (int i = 0; i < s.length(); i++)
```

```
hash = (R * hash + s.charAt(i)) % M;
```

- + Ogni carattere contribuisce in modo unico al risultato finale. Per avere una buona (cioè uniforme) distribuzione degli oggetti nella tabella (e avere collisioni distribuite) è conveniente utilizzare dei numeri primi per la base delle potenze e per la dimensione delle tabelle. Java usa $R=31$.

Chiavi composte

- + Se le chiavi sono campi multipli di interi, allora possiamo mescolarli come si fa con le stringhe.
- + Per esempio, supponiamo che le chiavi siano di tipo Data, che ha tre campi interi: day (2 cifre), month (2 cifre) e year (4 cifre). La funzione è:
- +
$$\text{int hash} = (((\text{day} * R + \text{month}) \% M) * R + \text{year}) \% M;$$

Java hashing

- + Java possiede per ogni tipo di dato un metodo chiamato `hashCode()` (che restituisce un intero a 32 bit).
- + L'implementazione di `hashCode()` per un oggetto deve essere consistente con `equals`. Cioè, se `a.equals(b)` è vero, allora `a.hashCode()` deve avere lo stesso valore numeric di `b.hashCode()`. Se i valori `hashCode()` sono uguali, questo non implica che gli oggetti siano uguali e, per verificare ciò bisogna usare il metodo `equals`.

Funzioni hash in Java

- + Tutte le classi Java ereditano un metodo `hashCode()` che restituisce un intero a 32 bit;
 - + E' necessario che
se `x.equals(y)` allora `(x.hashCode() == y.hashCode());`
 - + E' desiderabile che
se `!x.equals(y)` allora `(x.hashCode() != y.hashCode());`
- + Per default, viene restituito l'indirizzo di `x`.

Alcuni esempi

+ Integer:

```
public int hashCode ( ) { return value ; }
```

+ String: Metodo di Horner (L prodotti-accumuli) per il calcolo del polinomio: $h = s[0]31^{L-1} + \dots + s[L-1]31^0$:

```
public int hashCode ( ) {  
  int hash = 0;  
  for ( int i = 0; i < length ( ) ; i ++ )  
    hash = s [ i ] + ( 31 * hash ) ;  
  return hash ;  
}
```

Da hashCode a indice di array

+ hashCode(): intero tra -2^{31} e $2^{31} - 1$;

+ Funzione hash: intero tra 0 e $M - 1$.

```
private int hash (Key key ) {
```

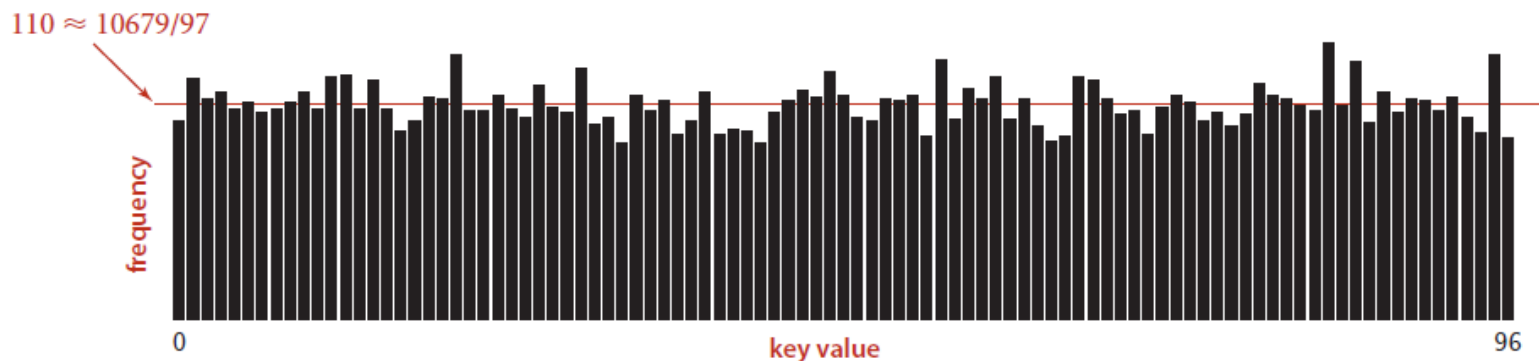
```
return ( key . hashCode ( ) & 0 x 7 f f f f f f f ) % M;
```

```
}
```

Maschera il bit con il segno, restituisce un numero naturale a 31 bit

Uniformità semplice

- + Il metodo `hashCode()` in Java ha come obiettivo l'uniformità
- + Per esempio, usando l'hashing standard sulle stringhe, con $M = 97$, la distribuzione delle frequenze dei valori hash per le parole del testo «Tales of Two Cities» di Charles Dickens



Hash value frequencies for words in *Tale of Two Cities* (10,679 keys, $M = 97$)

Implementazione in Java

- +

```
public Value get (Key key ) {  
    int i = hash ( key );  
    for (Node x = s t [ i ]; x != null ; x = x . next )  
        i f ( key . equals ( x . key ) ) return ( Value ) x . val ;  
    return null ;  
}
```
- +

```
public void put (Key key , Value val ) {  
    int i = hash ( key );  
    for (Node x = s t [ i ]; x != null ; x = x . next )  
        i f ( key . equals ( x . key ) ) { x . val = val ; return ; }  
    s t [ i ] = new Node( key , val , s t [ i ] );  
}
```
- + Nota: in SeparateChainingHashST.java si usa un array private `SequentialSearchST<Key, Value>[] st;`

Separate chaining: proprietà

- + Il numero di confronti per ricerca e inserimento è circa N/M ;
- + M troppo grande $\rightarrow$ troppe liste vuote;
- + M troppo piccolo $\rightarrow$ liste troppo lunghe;
- + Scelta tipica: $M = N/5 \rightarrow$ operazioni a tempo costante.
 - + Si può fare array resizing quando ci si discosta troppo da questo valore.

Tabelle dei simboli a confronto

algoritmo (struttura dati)	caso peggiore (dopo N inserzioni)		caso medio (dopo N inserzioni casuali)		operazioni ordinate efficienti?
	search	insert	search hit	insert	
ricerca seq. (lista non ordinata)	N	N	$N/2$	N	no
ricerca binaria (array ordinato)	$\log N$	$2N$	$\log N$	N	si
albero binario di ricerca (BST)	N	N	$1.39 \log N$	$1.39 \log N$	si
albero 2-3 (RBBST)	$2 \log N$	$2 \log N$	$1.00 \log N$	$1.00 \log N$	si
separate chaining	$\log N$	$\log N$	3 - 5	3 - 5	no

α tipico

Min, max,
etc tra le
chiavi

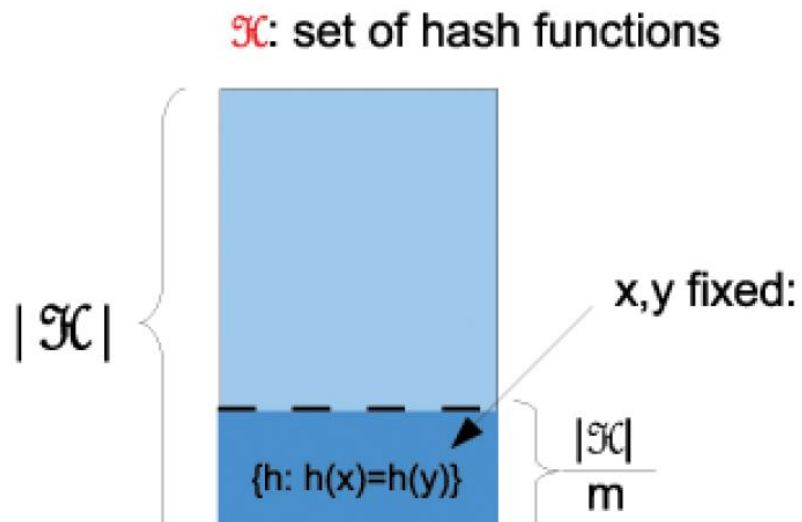
Hashing universale

- + Se un avversario sleale scegliesse le chiavi da elaborare con qualche funzione hash prefissata, potrebbe scegliere n chiavi che vengono mandate tutte nella stessa cella.
- + Un sistema efficace per ovviare a questo problema è scegliere casualmente la funzione hash, in modo che si indipendentemente dalle chiavi.
- + L'hashing universale consiste nel selezionare in modo casuale la funzione hash da una classe di funzioni accuratamente progettate.

Hashing universale

- + Una collezione $\mathcal{H}$ è detta universale se, per ogni coppia di chiavi distinte x e y , si ha che:

$$|\{h \in \mathcal{H} : h(x) = h(y)\}| \leq \frac{|\mathcal{H}|}{m}$$



Teorema: Se si considera una collezione universale di funzioni hash, utilizzata per inserire chiavi in una tabella hash con liste di collisione, la lunghezza media della lista è al massimo $1 + N/M$

L'hashing universale esiste?

- + Assumiamo che M sia primo e che le chiavi siano interi rappresentati con $\log_2 |U|$ bit, dove U è l'universo delle chiavi.
- + Prendiamo $R = \log_2 |U| / \log_2 M$ e assumiamo che sia un intero.
- + Decomponiamo i bit che compongono ogni chiave K in R parti, ovvero $K = [K_0, K_1, \dots, K_{R-1}]$. Ogni K_i è minore di M .
- + Per ogni intero A nell'universo $[0, \dots, |U|-1]$, facciamo la stessa operazione, ovvero $A = [A_0, A_1, \dots, A_{R-1}]$
- + Per ogni A , definiamo la funzione $h_A(K) = \sum_{i=0}^{R-1} A_i K_i \bmod M$
- + Si dimostra che la classe H di funzioni così definite è universale.
- + E' possibile trovare classi universali anche per valori di M che non siano un numero primo.

Indirizzamento aperto

Supponiamo di voler inserire un elemento con chiave k e la sua posizione "naturale" $h(k)$ sia già occupata.

L'indirizzamento aperto consiste nell'occupare un'altra cella, anche se potrebbe spettare di diritto a un'altra chiave.

Cerchiamo la cella vuota (se c'è) ispezionando altre celle secondo una sequenza di indici che dipende dalla chiave:

Deve essere una permutazione di
 $(0, 1, \dots, M-1)$

$c(k, 0), c(k, 1), c(k, 2), \dots, c(k, m-1)$

Pseudo codice

classe TavolaHashAperta **implementa** Dizionario:

dati: $S(m) = \Theta(m)$
un array v di dimensione m in cui ogni cella contiene una coppia $(elem, chiave)$.

operazioni:

insert($elem\ e, chiave\ k$)

1. **for** $i = 0$ **to** $m - 1$ **do**
2. **if** $(v[c(k, i)].elem = \text{null})$ **then**
3. $v[c(k, i)] \leftarrow (e, k)$
4. **return**
5. **errore** tavola piena

Vedremo in seguito la cancellazione

search($chiave\ k$) $\rightarrow elem$

1. **for** $i = 0$ **to** $m - 1$ **do**
2. **if** $(v[c(k, i)].elem = \text{null})$ **then**
3. **return** null
4. **if** $(v[c(k, i)].chiave = k)$ **then**
5. **return** $v[c(k, i)].elem$
6. **return** null

Scansione lineare o linear probing

- + I metodi di hashing ad indirizzamento aperto funzionano come la ricerca di parcheggio: se il posto è occupato, trova il prossimo posto libero e parcheggiati lì;
- + Si fa affidamento sulla presenza di spazi vuoti nella tabella hash di M posti, nella quale si memorizzano $N < M$ coppie chiave valore;
- + La scansione lineare (o linear probing) è il più semplice tra i metodi ad indirizzamento aperto, e in presenza di collisione cerca di occupare la cella immediatamente contigua;
- + Un probe (o ispezione) è un'operazione atta a determinare se un elemento della tabella ha chiave uguale o no a quella da cercare;
- + Invece di "sprecare" memoria per i riferimenti delle liste concatenate, si spreca memoria per tenere celle vuote nella tabella;

Metodi di ispezione: scansione lineare

$$c(k,i) = (h(k) + i) \bmod m$$

per $0 \leq i < m$

Esempio

	C			E			I			L			M			N			O			P			R			S			T			V		
	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30					
P																				P																
R																			P		R															
E							E												P		R															
C						C	E												P		R															
I						C	E					I							P		R															
P						C	E					I							P	P	R															
I						C	E					I	I						P	P	R															
T						C	E					I	I						P	P	R					T										
E						C	E	E				I	I						P	P	R					T										
V						C	E	E				I	I						P	P	R					T	V									
O						C	E	E				I	I						O	P	P	R				T	V									
L						C	E	E				I	I		L				O	P	P	R				T	V									
I						C	E	E				I	I	I	L				O	P	P	R				T	V									
S						C	E	E				I	I	I	L				O	P	P	R				S	T	V								
S						C	E	E				I	I	I	L				O	P	P	R				S	T	S	V							
I						C	E	E				I	I	I	L	I			O	P	P	R				S	T	S	V							
M						C	E	E				I	I	I	L	I	M		O	P	P	R				S	T	S	V							
E						C	E	E	E			I	I	I	L	I	M		O	P	P	R				S	T	S	V							
V						C	E	E	E			I	I	I	L	I	M		O	P	P	R				S	T	S	V	V						
O						C	E	E	E			I	I	I	L	I	M		O	P	P	R				S	T	S	V	V	O					
L						C	E	E	E			I	I	I	L	I	M		O	P	P	R				S	T	S	V	V	O	L				
M						C	E	E	E			I	I	I	L	I	M		O	P	P	R				S	T	S	V	V	O	L	M			
E						C	E	E	E	E		I	I	I	L	I	M		O	P	P	R				S	T	S	V	V	O	L	M			
N						C	E	E	E	E		I	I	I	L	I	M		O	P	P	R				S	T	S	V	V	O	L	M	N		
T						C	E	E	E	E		I	I	I	L	I	M		O	P	P	R				S	T	S	V	V	O	L	M	N	T	
E	E					C	E	E	E	E		I	I	I	L	I	M		O	P	P	R				S	T	S	V	V	O	L	M	N	T	

Inserimenti in tabella hash basata su indirizzamento aperto con scansione lineare delle lettere della parola:

PRECIPITEVOLISSI MEVOLMENTE

4,8 celle scandite in media per inserimento

PIÙ IN DETTAGLIO

key	hash	value	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
S	6	0							S									
E	10	1							0				E					
A	4	2					A		S				E					
R	14	3					2		0				1				R	
C	5	4					A	C	S				E				R	
H	4	5					2	5	0				1				3	
E	10	6					A	C	S	H			E				R	
X	15	7					2	5	0	5			6				3	X
A	4	8					A	C	S	H			E				R	X
M	1	9		M			8	5	0	5			6				3	7
P	14	10	P	M			A	C	S	H			E				R	X
L	6	11	10	9			8	5	0	5			6				3	7
E	10	12	P	M			A	C	S	H	L		E				R	X
			10	9			8	5	0	5	11		12				3	7

entries in red are new

entries in gray are untouched

keys in black are probes

probe sequence wraps to 0

keys[]

vals[]

Metodi di ispezione: scansione quadratica

La scansione quadratica reduce il problema dell'agglomerazione primaria

$$c(k,i) = (h(k) + c_1 i + c_2 i^2) \bmod M$$

per $0 \leq i < M$, c_1 e c_2 costanti e h funzione hash

Questo porta comunque ad una forma di addensamento (più lieve rispetto a quella primaria) detta agglomerazione secondaria

Metodi di scansione: hashing doppio

La scansione lineare o quadratica provocano effetti di agglomerazione, cioè lunghi gruppi di celle consecutive occupate che rallentano la scansione

L'hashing doppio riduce il problema

$$c(k,i) = \lfloor h_1(k) + i \cdot h_2(k) \rfloor \bmod m$$

per $0 \leq i < m$, h_1 e h_2 funzioni hash

	C					E				I				L				M				N				O				P				R				S				T				V																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																					
	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																				

Inserimenti in tabella hash basata su indirizzamento aperto con hashing doppio delle lettere della parola:

**PRECIPITEVOLISSI
MEVOLMENTE**

3,1 celle scandite in media per inserimento

Hashing doppio

- + Il valore di $h_2(k)$ deve essere coprimo con M perchè venga ispezionata tutta la tabella.
- + Un modo pratico per garantire questa condizione è scegliere M potenza di 2 e definire h_2 in modo che produca sempre un numero dispari.
- + Un altro modo è scegliere M primo e definire h_2 in modo tale che generi sempre un numero intero positivo minore di M . Per esempio:
 - + $h_1(k) = k \bmod M$
 - + $h_2(k) = 1 + (k \bmod M')$ con $M' < M$

Indirizzamento aperto: analisi

- + Si considera l'ipotesi di hashing uniforme, ovvero si suppone che ogni chiave abbia la stessa probabilità di avere come sequenza di probe una delle $M!$ permutazioni di $\{0,1,\dots,M-1\}$. Nel caso del separate chaining si suppone invece che ogni singolo valore abbia la stessa probabilità di essere prodotto.
- + TEOREMA: Nell'ipotesi di hashing uniforme, il numero atteso di probe per una ricerca senza successo è al più $1/(1-\alpha)$.
- + Dimostrazione: In una ricerca senza successo (search miss), ogni probe tranne l'ultimo accede ad una cella occupata che non contiene la chiave desiderata e l'ultima cella esaminata è vuota. Indichiamo con X la variabile casuale che rappresenta il numero di ispezioni fatte durante una ricerca. Per ogni i , sia q_i la probabilità che $X \geq i$.
Il numero atteso di probe è pari a $\sum_{i=0}^{\infty} q_i$.
La probabilità di fare almeno un probe è $q_1 = N/M = \alpha$. Per $i > 1$, la probabilità che l' i -esimo probe trovi una cella occupata dopo che i primi $i-1$ probe hanno trovato una cella occupata è $\frac{N-i+1}{M-i+1}$, quindi $q_i = \frac{N}{M} \frac{N-1}{M-1} \frac{N-2}{M-2} \dots \frac{N-i+1}{M-i+1} \leq \left(\frac{N}{M}\right)^i = \alpha^i$
Quindi il valore atteso di X è $\sum_{i=0}^{\infty} q_i \leq \sum_{i=0}^{\infty} \alpha^i = 1 + \alpha + \alpha^2 + \dots = \frac{1}{1-\alpha}$
- + Ciò significa che una prima ispezione è sempre effettuata, con una probabilità α si trova una cella occupata, quindi occorre una seconda ispezione. Con probabilità α^2 la cella è occupata...
- + In pratica: Il numero atteso di probe è non superiore a 3 se la tabella viene mantenuta a meno di metà della sua capienza.

Indirizzamento aperto: analisi

- + COROLLARIO: Nell'ipotesi di hashing uniforme, un'operazione di inserimento richiede in media al più $1/(1-\alpha)$ probe.
- + TEOREMA: Nell'ipotesi di hashing uniforme e che ogni chiave abbia la stessa probabilità di essere cercata, il numero atteso di probe per una ricerca con successo è al più $1/\alpha \ln(1/(1-\alpha))$.

Scansione lineare: osservazioni

- + La prima posizione esaminata determina l'intera sequenza di scansione; quindi abbiamo solo M sequenze di scansione distinte. Stessa considerazione vale per la scansione quadratica.
- + Il numero ottimo è $M!$ ed è dato dall'ipotesi di uniformità della funzione hash: ognuna delle $M!$ permutazioni di $\langle 0, \dots, m-1 \rangle$ è equiprobabile
- + Siamo molto lontani dal numero ottimo

Hashing doppio: osservazioni

- + La prima posizione esaminata è $h_1(k) \bmod M$; ogni posizione esaminata successivamente è distanziata dalla precedente di una quantità $h_2(k) \bmod M$.
- + La sequenza di scansione dipende da k in due modi: a seconda della chiave, possono variare sia la posizione iniziale che il passo
- + L'hashing doppio non soffre molto di fenomeni di agglomerazione
- + Ogni possibile coppia $(h_1(k), h_2(k))$ produce una sequenza di scansione distinta: abbiamo $O(M^2)$ sequenze di scansione distinte ed in questo senso è migliore della scansione lineare

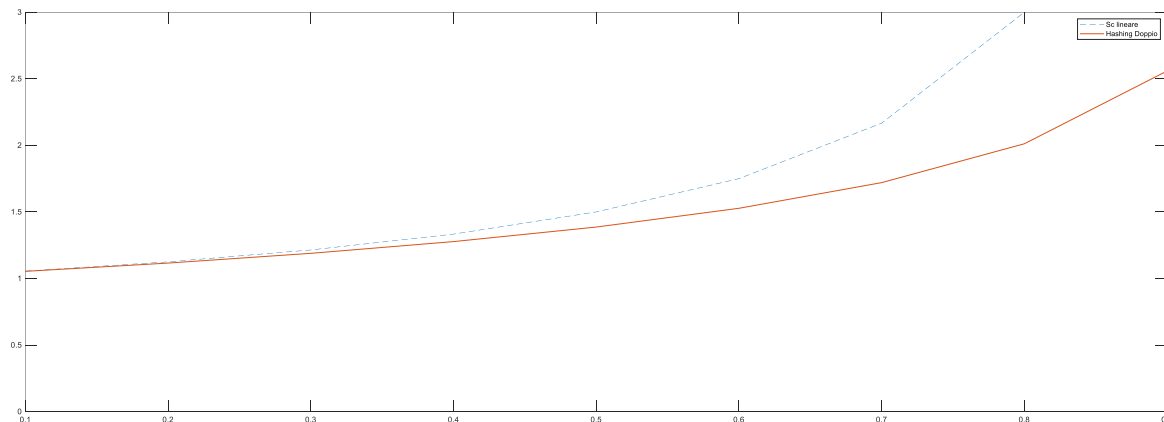
Analisi del costo di scansione

Nessuna di queste tecniche soddisfa l'ipotesi di hashing uniforme (la scansione lineare genera $\Theta(M)$ sequenze di ispezioni differenti, l'hashing doppio $\Theta(M^2)$).

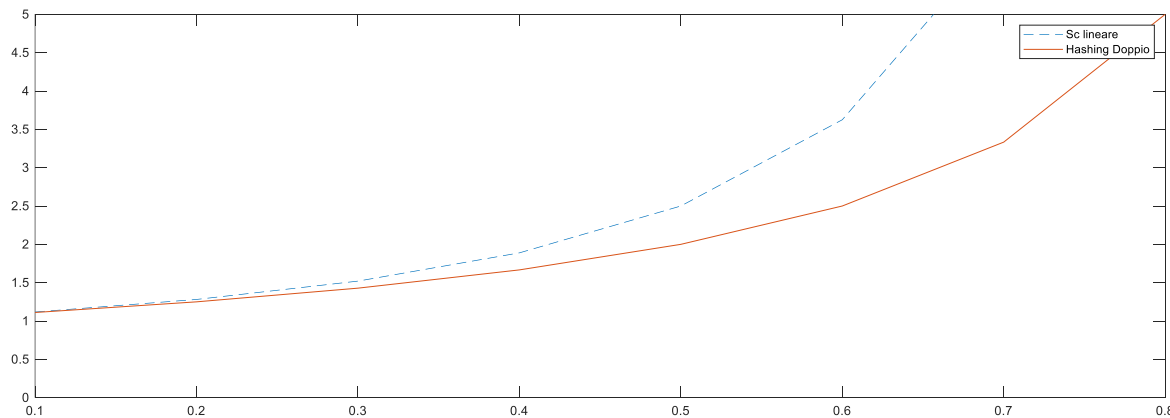
Si dimostra che il tempo richiesto in media da un'operazione di ricerca di una chiave, assumendo che le chiavi siano prese con probabilità uniforme da U :

<i>esito ricerca</i>	<i>sc. lineare</i>	<i>hashing doppio</i>
chiave trovata	$\frac{1}{2} + \frac{1}{2(1-\alpha)}$	$-\frac{1}{\alpha} \log_e(1 - \alpha)$
chiave non trovata	$\frac{1}{2} + \frac{1}{2(1-\alpha)^2}$	$\frac{1}{1-\alpha}$

Scansione lineare vs Hashing Doppio



Chiave trovata



Chiave non trovata

Tabelle dei simboli a confronto

algoritmo (struttura dati)	caso peggiore (dopo N inserzioni)		caso medio (dopo N inserzioni casuali)		operazioni ordinate efficienti?
	search	insert	search hit	insert	
ricerca seq. (lista non ordinata)	N	N	$N/2$	N	no
ricerca binaria (array ordinato)	$\log N$	$2N$	$\log N$	N	si
albero binario di ricerca (BST)	N	N	$1.39 \log N$	$1.39 \log N$	si
albero 2-3 (RBBST)	$2 \log N$	$2 \log N$	$1.00 \log N$	$1.00 \log N$	si
separate chaining	$\log N$	$\log N$	3 - 5	3 - 5	no
linear probing	$\log N$	$\log N$	3 - 5	3 - 5	no

Memoria occupata a confronto

method	space usage for N items (reference types)
<i>separate chaining</i>	$\sim 48 N + 64 M$
<i>linear probing</i>	between $\sim 32 N$ and $\sim 128 N$
<i>BSTs</i>	$\sim 56 N$